

CampusConnect

Full-Stack, Real-Time Campus Collaboration, Innovation & Micro-Gigs Marketplace

Stack: React Native (Expo) • TypeScript • Node.js • Express • PostgreSQL (Neon) • Prisma ORM • Socket.io

1. Executive Summary & Placement Pitches

30-Second HR Elevator Pitch

"CampusConnect is a production-ready, real-time campus collaboration and talent ecosystem built with React Native, TypeScript, Node.js, Express, PostgreSQL via Prisma ORM, and Socket.io. It bridges academic silos by allowing students to pitch cross-departmental projects, recruit teammates via an automated application lifecycle, earn bounties through micro-gigs, and collaborate in real-time with WebSockets. Built with enterprise security including JWT token rotation, 2-Factor Authentication, and optimistic UI updates, it demonstrates modern full-stack systems engineering."

2-Minute In-Depth Technical Pitch

"In university campuses, engineering, design, and business students operate in silos. Finding complementary talent for hackathons, capstones, and startups requires relying on informal chat groups where messages get lost. CampusConnect solves this through a dedicated multi-module platform:

1. An **Innovation Board** supporting project discovery, tech-stack filtering, and concurrent upvoting.
2. An **Application & Team Formation Engine** where accepting a team applicant automatically provisions a PostgreSQL Team record, updates roles, creates a private TEAM chat conversation, and broadcasts automated welcome messages and push notifications.
3. An **Internal Campus Gigs Marketplace** for peer-to-peer micro-tasks and bounties with stipend negotiation.
4. A **Real-Time WebSocket Engine** using Socket.io for typing indicators, bi-directional instant messaging, and live notifications.
5. A **Robust Security & Auth Architecture** supporting 2-Factor Authentication (OTP verification), bcrypt password hashing, and short-lived JWT access tokens paired with rotated refresh tokens in PostgreSQL.

I built this as a full-stack monorepo with 100% TypeScript type safety from database models to UI components, with passing integration test suites and automated database pooling."

2. System Architecture & Tech Stack Justification

Layer / Tool	Technology	Engineering Justification Over Alternatives
Mobile Client	React Native (Expo) + TS	Cross-platform codebase (iOS, Android, Web). Native 60fps performance without writing duplicate Swift & Kotlin.
State Store	Zustand	Lightweight, hook-based state management with zero Redux boilerplate and seamless Socket.io store integration.
Backend	Node.js + Express	Asynchronous, non-blocking event-driven runtime ideal for real-time WebSockets and concurrent queries.

Database	Neon PostgreSQL	ACID relational guarantees essential for recruitment applications, foreign keys with cascade deletions, and compound unique constraints.
ORM	Prisma 5	End-to-end type safety, auto-generated migrations, and native protection against SQL Injection attacks.
Real-Time	Socket.io	Bi-directional event channels, logical room broadcasting (conversation:id), and automatic HTTP polling fallback.
Security	JWT + Bcrypt + 2FA	Short-lived access tokens (15m) + database-persisted refresh token rotation (7d) and 6-digit OTP verification.

3. UI/UX Transformations & Design Decisions

Screen / Area	UI / UX Modernization Made	Placement Walkthrough Impact
Header & Nav	Branded title with emerald HUB badge; interactive user profile chip with initials; floating bottom tabs with active pill indicators.	Demonstrates mastery of mobile layout geometry and clean navigation flow hierarchy.
Ideas Board	Search box with icon & clear button; tech filter chips; author avatar pills; live pulsing dot status badges (■ Recruiting); optimistic upvote pills.	Exhibits optimistic UI rendering and complex list virtualization with instant user feedback.
Campus Gigs	Category filter pills with emojis (■, ■■, ■■); gold/amber stipend badges; duration estimates; clean application modal overlay.	Demonstrates responsive dialog styling, multi-filter querying, and marketplace ergonomics.
Chat Engine	Team group vs direct chat avatar icons; message preview snippets; asymmetric speech bubbles (chat-tail effect); typing indicator dot.	Shows real-time chat UX design matching Slack/WhatsApp campus community standards.
Alerts Feed	Categorized event badges (■, ■, ■, ■); soft light-emerald unread highlight; one-tap 'Mark all read' header action.	Proves understanding of event-driven notification systems with unread state tracking.
Student Profile	Hero card with verified student badge; academic metrics bar (skills count, interests, grad year); interactive GitHub & Portfolio link cards.	Showcases student resume & skill representation within a university network.

4. Complete Module & Function Reference

Module	Function & Route	Inputs	Internal Logic & Database Operations
Auth	register POST /api/auth/register	email, password, fullName, branch, gradYear	Zod validation; Bcrypt hashing (10 rounds); creates User & Profile in atomic transaction; returns JWT pair.
Auth	login POST /api/auth/login	email, password	Finds user by email; verifies password with bcrypt.compare; generates and persists rotated refresh token.
Auth	sendOtp POST /api/auth/send-otp	email	Generates random 6-digit OTP; stores in cache with 10-minute TTL; dispatches verification code.
Auth	verifyOtp POST /api/auth/verify-otp	email, code	Validates OTP code and expiration; invalidates code upon success to complete 2FA login.
Auth	refreshToken POST /api/auth/refresh-token	refreshToken	Verifies refresh token signature; queries RefreshToken table; deletes used token and creates new rotated pair.
Projects	getProjects GET /api/projects	search, branch, techStack, page, limit	Dynamic Prisma where filter; case-insensitive search; pagination (skip/take); computes hasUpvoted flag.
Projects	createProject POST /api/projects	title, summary, description, techStack	Zod schema validation; assigns ownerId from JWT; inserts project record into PostgreSQL.
Projects	toggleUpvote POST /api/projects/:id/upvote	params.id	Atomic query against compound unique key [projectId, userId]; deletes if existing, inserts if absent.
Teams	applyToProject POST /api/projects/:id/apply	params.id, note, contactLink	Validates applicant != owner; checks unique constraint; creates Application; emits live socket notification.
Teams	updateAppStatus PATCH /api/applications/:id/status	params.id, status	Core Flow: If ACCEPTED: updates app; creates Team; assigns OWNER/MEMBER roles; auto-creates TEAM Chat; injects welcome message; pushes live notification.
Gigs	getGigs / createGig GET & POST /api/gigs	title, stipend, skills, category	Queries open gigs with category filters; inserts new bounties; tracks student applicants.
Chat	getMessages GET /api/chat/:id/messages	params.id	Verifies participant authorization; fetches chronological messages; updates lastReadAt timestamp.
Chat	send_message WS event: send_message	conversationId, content	Persists message in PostgreSQL via Prisma; broadcasts to conversation room via Socket.io.
Notifs	getNotifications GET /api/notifications	Bearer Token	Retrieves notifications by userId ordered by date; calculates unread counter; supports mark-all-read.

5. Relational Database Design (Prisma Schema)

Model	Cardinality & Relations	Integrity Constraints & Indexing Strategy
User & Profile	User (1:1) Profile	email @unique. Profile cascades on user deletion (onDelete: Cascade).
RefreshToken	User (1:N) RefreshToken	token @unique. Stored hashed with expiration dates for instant session revocation.
ProjectUpvote	Project (1:N) ProjectUpvote	Compound Unique: @@unique([projectId, userId]) prevents duplicate votes.
Application	Project (1:N) Application	Compound Unique: @@unique([projectId, userId]) guarantees recruitment idempotency.

TeamMember	Team (1:N) TeamMember	projectId @unique on Team. @@unique([teamId, userId]) prevents duplicate roster entries.
Message	ChatConversation (1:N) Message	@@index([conversationId, createdAt]) accelerates chronological chat pagination.
Notification	User (1:N) Notification	@@index([userId, isRead]) optimizes unread badge count queries.

6. Placement Drive Technical Viva Question Bank (Top 15 Q&As;)

Q1: Why choose PostgreSQL over MongoDB for CampusConnect?

CampusConnect is inherently relational: users own projects, projects have teams, teams have members, and teams map 1-to-1 with chat conversations. PostgreSQL provides ACID compliance, foreign key cascade constraints, and compound unique constraints ([projectId, userId]) at the storage engine level, preventing duplicate votes or applications without complex distributed locks.

Q2: How does JWT Refresh Token Rotation protect against replay attacks?

With Token Rotation, each refresh token can only be used once. When a client requests a new access token, the server verifies the refresh token, deletes it from the database table, and issues both a new access token and a brand new refresh token. If an attacker intercepts a used token, subsequent reuse is immediately rejected, terminating the compromised session.

Q3: What is Optimistic UI and where did you implement it?

Optimistic UI immediately updates the client interface before the server responds, assuming success. In CampusConnect, I implemented this on the Project Upvote button. The upvote count increments and highlights immediately. If the server request fails, the client catches the error and reverts back to the original count.

Q4: How does Socket.io differ from standard native WebSockets?

Socket.io provides automatic fallback to HTTP long-polling if WebSockets are blocked by campus firewalls, heartbeat ping-pong monitoring, automatic reconnection with exponential backoff, and logical rooms (socket.join('conversation:id')) to broadcast messages exclusively to specific team participants.

Q5: How does the 2-Factor Authentication (OTP) workflow function?

When a student registers or signs in, primary credentials are validated first. The server generates a cryptographically random 6-digit OTP stored with a 10-minute TTL. The client transitions to the 2FA screen. Only when the student inputs the matching code does the backend issue the final JWT access and refresh token pair.

Q6: What happens in the database when a project application is accepted?

It executes an atomic orchestration: updates Application.status = 'ACCEPTED', finds or creates a Team record, assigns the owner as OWNER and applicant as MEMBER in TeamMember, auto-provisions a TEAM chat conversation, registers both participants, injects a system welcome message, and emits a real-time notification to user:\${applicantId}.

Q7: How do you prevent SQL Injection and XSS attacks?

SQL Injection is prevented by using Prisma ORM, which parameterizes all SQL queries automatically. XSS is mitigated by React Native, which does not execute raw HTML strings inside native views, and by validating all incoming request bodies against strict Zod schemas.

Q8: How does the mobile client handle authenticated requests?

We configure an Axios HTTP client with request and response interceptors. The request interceptor attaches Authorization: Bearer . The response interceptor listens for 401 Unauthorized errors, calls /api/auth/refresh-token, updates the stored token in Zustand, and retries the original request seamlessly.

Q9: How would you scale CampusConnect to 100,000 active students across 50 colleges?

1. Multi-Tenancy: Add an indexed collegeId column across tables. 2. Database Scaling: Introduce PostgreSQL read replicas for read-heavy feeds. 3. WebSocket Scaling: Attach @socket.io/redis-adapter with Redis Pub/Sub to broadcast events across multiple clustered Node.js processes. 4. Caching: Place Redis in front of frequently queried project feeds. 5. CDN: Store student media in Amazon S3 with CloudFront.

Q10: What indexing strategies are used in your schema and why?

Targeted B-tree indexes: `@@index([ownerId])` on Project for user portfolio filtering, `@@index([conversationId, createdAt])` on Message for chronological chat pagination, and `@@index([userId, isRead])` on Notification to calculate unread counters in $O(\log N)$ time.

Q11: Explain how compound unique constraints solve business problems in your database.

In a mobile app, users might repeatedly tap 'Upvote' or 'Apply'. By defining `@@unique([projectId, userId])` on ProjectUpvote and Application, the database physically rejects duplicate records, guaranteeing idempotency at the storage engine level.

Q12: How did you design the state management in the React Native application?

I used Zustand. It provides lightweight stores with minimal boilerplate: `useAuthStore` manages user profile, tokens, and device accounts; `useSocketStore` manages the persistent WebSocket connection lifecycle across screens without prop-drilling.

Q13: What HTTP status codes do your REST APIs return and why?

200 OK for queries and updates, 201 Created for creating projects/applications, 400 Bad Request for Zod validation errors, 401 Unauthorized for missing/expired tokens, 403 Forbidden for unauthorized mutations, 404 Not Found for missing resources, and 500 handled by centralized error middleware.

Q14: What was the most challenging bug you encountered and how did you resolve it?

During team formation on applicant acceptance, concurrent approvals could spawn duplicate team chat rooms. I resolved this by enforcing `projectId String @unique` on the Team model and implementing an upsert/find-or-create pattern so only one team and one group chat ever exists per project.

Q15: What are your key takeaways from building this project?

Building CampusConnect taught me how to architect an end-to-end full-stack product. I gained hands-on experience in atomic relational transactions with Prisma, optimistic client-side UI updates, real-time WebSocket room broadcasting, and enterprise auth with token rotation and 2FA.